minGPT: Adım Adım ve FPGA Potansiyeli · konuşma metni

Nasıl kullanılır: Her başlık bir slayta karşılık gelir; numaralar slayt numarasıdır. Aynı metin, sunumda her slaytın konuşmacı notunda da var. Toplam yaklaşık 6 100 kelime: rahat bir tempoyla 45 dakika civarı. Süre kısaysa her bölümün ilk iki-üç cümlesini söyleyip slayttaki şekli göstermek yeterli. Metnin sonunda hocanın sorabileceği sorular ve kısa cevapları var.

Akış: 1–2 giriş · 3–5 temel kavramlar · 6–9 minGPT nedir · 10–21 minGPT adım adım (12–18 attention) · 22–23 generation ve KV cache · 24–27 FPGA potansiyeli · 28 özet.

---

1. minGPT: adım adım

Hocam merhaba. Bitirme projemin ilk bölümünü anlatacağım. Projenin bütününde Karpathy'nin minGPT'sini ve aynı soydan gelen modelleri bir FPGA üzerinde çalıştırıyoruz. Ama bunu yapabilmek için önce modelin içinde tam olarak ne olduğunu anlamamız gerekiyordu. Bu sunum o yüzden yalnızca minGPT'ye ayrıldı. Bir language model'in ne yaptığından başlayacağım, minGPT'nin bir token'ı nasıl işlediğini her adımda formülüyle, şekliyle ve onu yapan kod satırıyla göstereceğim. Sonunda da bu yapının FPGA açısından nerede güçlü, nerede zorlayıcı olduğunu tartışacağım. Kendi donanım tasarımımızın ayrıntılarına bu sunumda girmiyorum; onu ayrı bir bölümde anlatacağım. Bir not: teknik terimleri bilerek İngilizce bıraktım, çünkü literatürde ve kodda bu isimlerle geçiyorlar; Türkçe karşılıklarını kullanırsam kodla eşleştirmek zorlaşıyor.

2. Bu sunum minGPT'yi sıfırdan kuruyor ve sonunda FPGA açısından değerlendiriyor

Sunumun akışı beş adım. Önce temel kavramları kuracağım: language model ne demek, token ve embedding ne, bir modelin iyi olduğunu nasıl ölçüyoruz. Bunlar olmadan minGPT'nin kodu anlamsız kalır. İkinci olarak minGPT'nin ne olduğunu konumlandıracağım: yeni bir mimari değil, GPT-2'nin sade bir yeniden yazımı. Modern LLM'lerden farkını ve 300 satırlık bir kodun nasıl olup da çalışan bir GPT olabildiğini anlatacağım. Üçüncü ve en uzun kısım minGPT'nin bir token'ı adım adım nasıl işlediği. Burada attention'a özellikle zaman ayırdım; önce hangi problemi çözdüğünü, sonra fikrini, sonra formülünü, elle hesaplanmış bir örneği ve en sonda bizim eğittiğimiz modelin gerçekte ne yaptığını göstereceğim. Dördüncü olarak modelin metni nasıl ürettiğini ve KV cache

fikrini anlatacağım; bu, donanıma geçişin anahtarı. Son olarak bu yapının FPGA açısından güçlü ve zayıf yanlarını tartışacağım.

3. Language model, geçmişe bakarak bir sonraki token'ın olasılık dağılımını tahmin eder

Önce en temel soru: language model ne yapar? Bir metnin, yani bir token dizisinin olasılığını verir. Bunu doğrudan hesaplamak imkânsız gibi görünür, ama olasılık kuramındaki zincir kuralı işi basitleştiriyor. Bir dizinin olasılığı, her adımda "şimdiye kadar görülenlere göre bir sonraki token'ın olasılığı"nın çarpımına eşittir. Burada önemli olan şu: bu bir yaklaşıklık değil, tam bir eşitlik. Dolayısıyla bir language model'in öğrenmesi gereken tek şey bir fonksiyon: geçmişi al, bir sonraki token için bir olasılık dağılımı ver. ChatGPT'den minGPT'ye kadar bütün GPT tarzı modeller tam olarak bunu yapar. Sağdaki örnek isim üreten bir modelden. Model "e, m, m" harflerini görmüş; şimdi dördüncü harf için 27 sembolün her birine bir olasılık veriyor. Değerler temsilî, ama mantık gerçek: "a" harfine yüksek olasılık verir, çünkü "emma" sık görülen bir isim. Metin üretmek de buradan çıkar: bu dağılımdan bir harf seçeriz, dizinin sonuna ekleriz ve aynı işlemi tekrarlarız. Her yeni token bir öncekinin üzerine kurulduğu için buna autoregressive generation denir. Bu sunumun geri kalanında hep bu tek işlemin, yani "geçmişten sonraki token dağılımını hesaplama"nın içini açacağım. Donanım açısından da işin özü bu: bir çağrıda bir token girer, sonraki token için bir dağılım çıkar.

4. Token bir sembolün sayı kimliğidir; embedding bu kimliği öğrenilmiş bir vektöre çevirir

Model metni doğrudan göremez; önce metni token'lara bölmemiz gerekir. Token, bir sembolün sayı kimliğidir. İsim verisinde her harf bir token: nokta 0, a 1, b 2 diye gider, e 5, m 13. Ama bu sayılar yalnızca etiket. m'nin 13 olması, e'nin iki buçuk katı olduğu anlamına gelmez. Bu sayıları doğrudan bir ağa verirsek ağ olmayan bir büyüklük ilişkisi öğrenmeye çalışır. Çözüm embedding: her token için C tane sayıdan oluşan bir vektör tutarız. Bütün bu vektörler V satır, C sütunluk bir tabloda durur. Token id'si tablonun hangi satırını okuyacağımızı söyler. Bu vektörlerin değerleri de diğer weight'ler gibi training sırasında öğrenilir. Benzer bağlamlarda geçen token'ların vektörleri birbirine yaklaşır; örneğin sesli harflerin vektörleri birbirine benzemeye başlayabilir. Yani id'nin taşıyamadığı "benzerlik" bilgisini vektör taşır. Matematiksel olarak embedding, token'ın one-hot vektörüyle tablonun çarpımıdır. One-hot vektörde yalnızca bir tane 1 olduğu için bu çarpım tablonun o satırını seçmekten ibarettir. Bu yüzden pratikte embedding bir hesap değil, bir tablo okuması. minGPT kodunda bunu nn.Embedding yapar: içine token id'leri verirsiniz, her biri için C boyutlu vektörü döner. Bu ayrıntı FPGA açısından önemli; tablo okuması, donanımda bir bellekten tek adres okumak demek. Hiç çarpma gerekmez.

5. Model, doğru token'a verdiği olasılıkla ölçülür (cross-entropy loss) ve bu loss'u küçültecek şekilde eğitilir

Bir modelin iyi olup olmadığını nasıl ölçeriz? Her pozisyonda modelin doğru token'a verdiği olasılığa bakarız. Model doğru cevaba yüksek olasılık veriyorsa iyidir. Bu olasılığın eksi doğal logaritmasını alıp bütün pozisyonlarda ortalarız; buna cross-entropy loss denir. Logaritma doğal olduğu için birimi nat/token. Tablodaki satırlar sezgi veriyor. Hiçbir şey bilmeyen bir model 27 harfe eşit olasılık verir; doğru harfe 1/27 düşer, loss ln 27, yani 3,30 olur. Model iki aday arasında kararsızsa, yani doğruya 0,5 veriyorsa, loss 0,69. Doğruya 0,9 veriyorsa 0,11. Mükemmel bir model sıfır loss alır. Logaritma küçük olasılıkları çok sert cezalandırır: model emin olup yanılırsa loss çok büyür. Training de bu sayıyı küçültmek demek. Sağdaki dört satır minGPT'nin training döngüsünün kalbi. Birinci satırda model tahminini yapar ve loss'u hesaplar. İkinci satırda eski gradient'ler sıfırlanır. Üçüncü satırda backward çağrısıyla PyTorch, loss'un her weight'e göre türevini, yani gradient'ini otomatik hesaplar. Dördüncü satırda optimizer, burada AdamW, her weight'i loss'u azaltacak yönde küçük bir adım kaydırır. Bu döngü binlerce kez tekrarlanır. Bir ayrımı baştan netleştirmek istiyorum: training tamamen bilgisayarda, floating point sayılarla yapılıyor. FPGA tarafında hiçbir zaman training yapmıyoruz; yalnızca eğitilmiş modeli çalıştırıyoruz. Buna inference denir. Bu ayrım önemli, çünkü inference'ın hesabı training'inkinden çok daha basit: ne gradient ne optimizer var, sadece ileri yönde hesap.

6. minGPT'nin tamamı üç parça: embedding, L tane aynı block ve bir output katmanı

Şimdi minGPT'nin kendisine geçiyorum. Bütün model aslında bu tek satırlık şemadan ibaret. Soldan sağa okuyalım. Önce embedding: token'ın vektörü ile pozisyonunun vektörü toplanır. Sonra L tane block gelir. Bütün block'lar aynı yapıda, sadece weight'leri farklı. Her block iki alt adım yapar. Birincisi attention: her token, kendisinden önceki token'lardan bilgi toplar. Bu modelin token'lar arasında bilgi taşıyan tek yeri. İkincisi MLP: her token kendi vektörünü kendi içinde işler, diğer token'lara bakmaz. İki alt adımın da önünde bir LayerNorm var ve ikisinin de sonucu x'in yerine geçmez, x'in üstüne eklenir; buna residual connection denir. Block'lardan sonra son bir LayerNorm ve lm_head adlı bir katman gelir. lm_head, vocabulary'deki her token için bir skor üretir; bunlara logit diyoruz. Logit'ler softmax'tan geçince bir sonraki token'ın olasılık dağılımı olur. Soldaki kod GPT.forward fonksiyonunun kendisi. Şemadaki her kutunun bir satıra karşılık geldiğini görüyorsunuz: wte token embedding, wpe position embedding, for döngüsü block'lar, ln_f son norm, lm_head çıkış. Bu sunum boyunca kullanacağım model minGPT'nin en küçük hazır yapılandırması olan gpt-nano: 3 block, her block'ta 3 attention head, vektör boyutu 48, head başına 16 boyut. Vocabulary sadece 3 token, çünkü görevimiz 0, 1, 2 rakamlarını sıralamak; context en fazla 11 token. Toplam yaklaşık 85,7 bin parametre. Şimdi bu şemadaki her

kutuyu sırayla açacağım; ama önce minGPT'nin ne olduğunu ve neden bu kadar kısa olduğunu konumlandırayım.

7. minGPT yeni bir mimari değil: GPT-2'nin eğitim amaçlı, sade bir yeniden yazımı

minGPT'nin ne olduğunu doğru konumlandırmak gerekiyor, çünkü "minGPT diğer LLM'lerden ne farklı yapıyor" sorusunun cevabı biraz şaşırtıcı: mimari olarak hiçbir şeyi farklı yapmıyor. Andrej Karpathy minGPT'yi 2020'de, GPT'nin nasıl çalıştığını göstermek için yazdı. Mimari, OpenAI'nin 2019'daki GPT-2'siyle birebir aynı. Bunun en güçlü kanıtı şu: minGPT'de from_pretrained diye bir fonksiyon var ve OpenAI'nin yayımladığı GPT-2 weight'lerini olduğu gibi yükleyip çalıştırabiliyor. Mimari bir parametre bile farklı olsaydı bu mümkün olmazdı. Peki modern LLM'ler, örneğin Meta'nın Llama 2'si ya da Karpathy'nin daha yeni nanochat'i, neyi farklı yapıyor? Tablo bunu özetliyor. Normalization'da LayerNorm yerine RMSNorm kullanıyorlar; ortalamayı çıkarmadan yalnızca büyüklüğe göre ölçekliyor, biraz daha ucuz. Pozisyon bilgisini ayrı bir tablodan eklemek yerine RoPE ile veriyorlar; q ve k vektörlerini pozisyona bağlı bir açıyla döndürüyor. MLP'de GELU yerine SwiGLU ya da ReLU kare kullanıyorlar. Attention'da GQA kullanabiliyorlar: birkaç query head'i aynı key ve value'yu paylaşıyor, bu da bellek tasarrufu sağlıyor. Bias'ları kaldırıyorlar. Generation sırasında KV cache kullanıyorlar; minGPT'nin generate fonksiyonu bunu yapmıyor, her adımda bütün diziyi baştan hesaplıyor. Ve tabii ölçek: gpt-nano 85 bin parametre, modern modeller milyarlarca. Ama asıl mesaj alttaki kutuda: çekirdek mekanizma aynı. Hepsi decoder-only transformer; embedding, tekrarlanan block'lar, her block'ta attention ve MLP, residual connection. Modern LLM'ler bu çekirdeğin ince ayarlı ve çok büyütülmüş hâli. Bu yüzden minGPT'yi baştan sona anlamak, hepsinin temelini anlamak demek. Bizim için seçim sebebi de bu: küçük, okunabilir ve tamamen anlaşılabilir.

8. 310 satır neden yetiyor? Ağır işi PyTorch yapıyor, transformer da kendini tekrar ediyor

Akla gelen doğal soru şu: milyar dolarlık modellerin temeli olan bir mimari nasıl oluyor da yorumlarıyla birlikte 310 satırlık bir dosyaya sığıyor? Bunun beş sebebi var ve beşi de koddan okunabiliyor. Birincisi: işin ağır kısmını PyTorch yapıyor. En zor kısım olan backpropagation, yani her weight için gradient hesabı, PyTorch'un autograd özelliğiyle otomatik. minGPT'de bu tek satır: loss.backward(). Linear katman, LayerNorm, softmax, AdamW optimizer, GPU kernel'leri; hepsi hazır. minGPT yalnızca bunların nasıl dizileceğini tanımlıyor. İkincisi: transformer kendini tekrar ediyor. Tek bir Block sınıfı yazılıyor ve ModuleList ile L kez kopyalanıyor. 3 block da olsa 96 block da olsa kod aynı. Üçüncüsü: query, key ve value için üç ayrı katman yazmak yerine tek bir Linear katman üçünü birden hesaplıyor; çıkış üçe bölünüyor. Dördüncüsü bence en şık hilesi: birden çok attention head

için döngü yazılmıyor. Tensor'ün boyutları view ve transpose ile yeniden düzenleniyor; head boyutu batch boyutu gibi davranıyor ve bütün head'ler tek bir matris çarpımıyla hesaplanıyor. Beşincisi training'in verimliliğiyle ilgili: causal mask sayesinde bir dizinin her pozisyonu yalnızca kendinden öncekilere bakıyor. Bu yüzden T uzunluğundaki bir dizide T tane "sonraki token" tahmini tek bir forward geçişinde, paralel olarak yapılabiliyor. Sağdaki kutu da madalyonun öbür yüzü: minGPT'de bilerek olmayanlar. KV cache yok, FlashAttention gibi hızlı kernel'ler yok, mixed precision yok, birden çok GPU ile training yok. Hepsi kodu uzatırdı. Amaç hız değil, okunabilirlik. Bu bizim için iyi bir başlangıç noktası, ama FPGA'ye taşırken bu eksiklerin bazılarını, özellikle KV cache'i, kendimiz eklememiz gerekecek.

9. Çalışan örneğimiz: gpt-nano altı rakamı küçükten büyüğe sıralamayı öğreniyor

Bu sunumdaki bütün örnekleri tek bir görev üzerinden vereceğim. Bu görev minGPT'nin kendi demo görevi: 0, 1 ve 2 rakamlarından oluşan altı rakamlık bir diziyi küçükten büyüğe sıralamak. Mesela girdi 2 0 1 1 0 2 ise beklenen çıktı 0 0 1 1 2 2. Burada ilginç olan, modelin sıralama algoritması bilmemesi. Model yalnızca bir language model; yani girdiyi ve çıktıyı yan yana koyduğumuz 12 token'lık diziyi görüyor ve her pozisyonda bir sonraki token'ı tahmin etmeyi öğreniyor. Soldaki kod eğitim örneğinin nasıl kurulduğunu gösteriyor. Girdi sıralanıyor, ikisi uç uca ekleniyor, 12 token oluyor. Modele ilk 11 token veriliyor; hedef ise bir kaydırılmış dizi, yani her pozisyonda bir sonraki token. Son satır önemli: girdi kısmındaki hedefler eksi bir yapılıyor. Bu, "burada loss hesaplama" demek. Çünkü girdinin bir sonraki rakamını tahmin etmek anlamsız, girdi rastgele. Model yalnızca çıktı kısmında, yani sıralı rakamları yazarken değerlendiriliyor. Bu görevi seçmemizin büyük bir avantajı var: olası girdi sayısı 3 üzeri 6, yani yalnızca 729. Örneklem almak yerine modeli bütün girdi uzayında test edebiliyoruz. Eğitilmiş gpt-nano 729 girdinin hepsini doğru sıralıyor. Şimdi bu modelin bir token'ı nasıl işlediğine adım adım bakalım.

10. Adım 1 · Token embedding ile position embedding toplanır: vektör hem "ne" hem "nerede" bilgisini taşır

Birinci adım embedding. Her pozisyon için iki vektörü topluyoruz. Birincisi token embedding: o pozisyondaki token'ın vektörü, yani "bu token ne?" sorusunun cevabı. İkincisi position embedding: pozisyonun vektörü, yani "bu token dizinin neresinde?" sorusunun cevabı. İkisi de öğrenilen tablolardan okunuyor; token embedding tablosunda vocabulary boyutu kadar, position embedding tablosunda en uzun context kadar satır var. İki vektör eleman eleman toplanıyor ve sonuç block'lara giriyor. Şekildeki renkler sayıları temsil ediyor; amaç, iki bilginin tek bir vektörde birleştiğini göstermek. Peki position neden gerekli? Biraz sonra göreceğimiz attention, geçmişteki vektörlerin ağırlıklı bir toplamı.

Toplama işlemi sıra bilmez: a artı b, b artı a ile aynı. Pozisyon bilgisi eklenmeseydi model “2 0 1” ile “1 0 2” dizilerini birbirinden ayırt edemezdi; sıralama gibi sırayla ilgili bir görevi hiç öğrenemezdi. Position embedding sayesinde aynı “0” token’ı birinci pozisyonda ve beşinci pozisyonda farklı vektörlerle temsil ediliyor. Kodda bu dört satır: arange ile 0’dan t’ye pozisyon indeksleri üretiliyor, iki tablodan vektörler okunuyor ve toplanıyor. Toplamın etrafındaki drop, dropout; training sırasında rastgele bazı elemanları sıfırlayarak ezberlemeyi önlüyor. Inference’ta ise hiçbir şey yapmıyor, yani modeli çalıştırırken bu satırı yok sayabiliriz. Bir not: modern modeller pozisyonu ayrı bir tabloyla değil, RoPE ile veriyor; minGPT’nin yolu daha basit, bir tablo okuması.

11. Adım 2 · LayerNorm, her token vektörünü ortalaması 0 ve varyansı 1 olacak şekilde yeniden ölçekler

İkinci adım LayerNorm. Her token’ın vektörünü kendi içinde normalize ediyoruz: vektörün elemanlarının ortalamasını çıkarıyor, standart sapmasına bölüyoruz. Böylece her vektörün ortalaması sıfır, varyansı bir oluyor. Ardından öğrenilen iki parametre uygulanıyor: γ ile eleman başına çarpıyor, β ile kaydırıyoruz; yani model gerekirse normalizasyonun etkisini kısmen geri alabiliyor. Neden gerekli? Derin bir ağda vektörler her layer’dan geçtikçe büyüklükleri kontrolsüz biçimde büyüyüp küçülebilir. Bu da training’i kararsız yapar. LayerNorm, her alt adımın girişinde ölçeği sabitliyor. Sağdaki küçük örnek: [2, 4, 6] vektörünün ortalaması 4, standart sapması yaklaşık 1,63. Ortalamayı çıkarıp standart sapmaya bölünce yaklaşık [−1,22, 0, 1,22] oluyor; çubuklar sıfır çizgisinin iki yanına simetrik dağılıyor. minGPT, GPT-2 gibi “pre-LN” düzenini kullanıyor: LayerNorm, attention’dan ve MLP’den önce uygulanıyor. Koddaki son iki satır bunu gösteriyor: önce ln_1, sonra attention; önce ln_2, sonra MLP. Dikkat edilecek nokta, residual yolun kendisinin normalize edilmemesi; normalize edilen kopya alt adıma giriyor, sonuç orijinal x’e ekleniyor. FPGA açısından bir not düşmek istiyorum: LayerNorm’da ortalama, karekök ve bölme var. Toplama ve çarpma donanımda ucuz ama karekök ve bölme pahalı ve yavaş. Bu adımı FPGA potansiyeli bölümünde tekrar anacağım.

12. Attention neden gerekli? Her token’ın, önceki token’lardan bilgi toplaması gerekiyor

Sıra modelin en önemli parçasına, attention’a geldi. Formüle geçmeden önce hangi problemi çözdüğünü anlatmak istiyorum. Embedding’ten sonra her pozisyonda bir vektörümüz var, ama bu vektör yalnızca kendi token’ını ve pozisyonunu biliyor. Bir sonraki token’ı tahmin etmek için ise geçmişe bakmak gerekiyor. Sıralama görevinde çıktıyı yazarken model girdideki rakamlara bakmak zorunda; bir isimde de sonraki harf, önceki harflere bağlı. Yani her pozisyonda geçmişteki vektörlerden bilgi toplayıp tek bir vektörde birleştirmemiz gerekiyor. Diyelim ki dördüncü pozisyondayız ve elimizde x0, x1, x2, x3 var. Bunlardan tek

bir y3 vektörü nasıl elde ederiz? En basit fikir, üstteki kutu: hepsinin ortalamasını almak. Her birine 0,25 ağırlık. Bu bir şey öğrenir ama ciddi bir sınırı var: model neye bakacağını seçemez; önemli token da önemsiz token da aynı ağırlığı alır. Attention'ın fikri alttaki kutu: yine bir ağırlıklı ortalama alıyoruz, ama ağırlıklar sabit değil. Model, token'ların içeriğine bakarak hangisinin şu an önemli olduğuna kendisi karar veriyor. Örnekte ağırlığın 0,70'i x1'e gidiyor; çünkü model x1'in bu tahmin için en ilgili token olduğuna karar vermiş. Değerler temsilî, ama birazdan bizim modelimizden gerçek değerler göstereceğim. Alttaki tanımı vurgulamak istiyorum, çünkü bütün attention bu tek cümleden ibaret: attention, geçmişteki vektörlerin, ağırlıkları içeriğe göre hesaplanan bir ağırlıklı ortalamasıdır. Ağırlıklar her zaman pozitif ve toplamları bir. Geri kalan her şey, yani query, key, value, softmax, bu ağırlıkların nasıl hesaplandığıyla ilgili.

13. Attention'ın fikri: bir sözlükte arama yapmanın "yumuşak" bir versiyonu

Ağırlıkları içeriğe göre nasıl hesaplarız? Bunun için en iyi benzetme bir sözlükte arama yapmak. Soldaki kutu sıradan bir Python dict'i. Her key'in bir value'su var. d["kedi"] dediğimizde aradığımız şey, yani query, tam olarak bir key ile eşleşiyor ve yalnızca o key'in value'su dönüyor. Eşleşen key'in ağırlığı 1, diğerlerinin 0. Attention bu aramanın yumuşak bir versiyonu. Sağdaki kutuya bakalım. Query yine var, key'ler ve value'lar yine var; ama tam eşleşme aramıyoruz. Query'yi her key ile karşılaştırıp bir benzerlik puanı hesaplıyoruz; bu puan iki vektörün dot product'ı. Sonra bu puanları softmax ile pozitif ve toplamı 1 olan ağırlıklara çeviriyoruz. Sonuç, value'ların bu ağırlıklarla alınmış karışımı. Örnekte query en çok k0'a benziyor, ağırlığın 0,70'i v0'a gidiyor; ama v1 ve v2 de az da olsa katkı yapıyor. Yumuşak aramanın büyük avantajı türevlenebilir olması: sert aramada "hangi key" sorusunun türevi yok, ama yumuşak aramada her ağırlık sürekli değişiyor. Bu yüzden model, training sırasında neye bakacağını gradient ile öğrenebiliyor. Alttaki üç kutu üç kavramı özetliyor. Query, şu anki token'ın "ne arıyorum" sorusu. Key, her token'ın "bende ne var" diye kendini tanıtan etiketi. Value ise o token seçilirse aktaracağı içerik. Kodun üç satırı da attention'ın neredeyse tamamı. Bir sonraki slaytta bu üç vektörün nereden geldiğini, sonra da gerçek formülü göstereceğim.

14. Adım 3 · Query, key ve value aynı vektörden, öğrenilen üç linear projection ile üretilir

Üçüncü adım: query, key ve value vektörlerini üretmek. Bunlar dışarıdan gelmiyor; her token kendi vektöründen üretiyor. Normalize edilmiş 48 boyutlu vektör, c_attn adlı bir Linear katmandan geçiyor. Bu katmanın 144 çıkışı var; bunlar 48'erli üç parçaya bölünüyor: birinci parça query, ikinci key, üçüncü value. Yani aslında üç ayrı öğrenilmiş projection var, minGPT verimlilik için bunları tek bir matriste birleştirmiş. Sonuçta her token'ın kendi

query'si, key'i ve value'su oluyor. Query, o token'ın geçmişte ne aradığını; key, diğer token'lara kendini nasıl tanıttığını; value ise seçilirse ne aktaracağını temsil ediyor. Bu isimler bir benzetme; matematikte yalnızca üç öğrenilmiş linear projection var ve hangi rolü üstlenecekleri training sırasında ortaya çıkıyor. Burada Linear katmanı da tanımlamak istiyorum, çünkü modelin her yerinde karşımıza çıkacak. Linear katman bir matris-vektör çarpımı artı bir bias. Her çıkış elemanı, girişlerin öğrenilmiş weight'lerle ağırlıklı toplamı. Bu tek satır, her token için 144 çarpı 48, yani 6 912 çarpma-toplama yapıyor. Çarpma-toplamaya kısaca MAC diyoruz: multiply-accumulate. Modelin hesabının büyük çoğunluğu bu tür Linear katmanlar; bunu FPGA bölümünde sayılarla göstereceğim, çünkü bir donanım tasarımının neye odaklanması gerektiğini tam olarak bu belirliyor. Kodda iki satır var: tanımda tek bir Linear, forward'da çıkışın split ile üçe bölünmesi.

15. Adım 4 · Scaled dot-product attention üç işlemdir: score, softmax ve value'ların ağırlıklı toplamı

Şimdi attention'ın gerçek formülü. Adı scaled dot-product attention ve üç işlemden oluşuyor. Birinci işlem score. t pozisyonundaki token'ın query'sini, geçmişteki her j pozisyonunun key'i ile dot product'a sokuyoruz. İki vektör aynı yöne bakıyorsa dot product büyük, farklı yönlere bakıyorsa küçük ya da negatif. Yani score, "bu geçmiş token, şu an aradığım şeye ne kadar benziyor" sorusunun cevabı. j küçük eşit t koşulu, yalnızca geçmişe bakıldığını söylüyor; bunu birazdan maske ile göreceğiz. Score'u $\sqrt{d_h}$'ye, yani head boyutunun karekökünе bölüyoruz. Bunun nedeni şu: q ve k'nin bileşenleri yaklaşık bağımsız ve varyansları bir ise, dot product d_h tane çarpımın toplamı olduğu için varyansı d_h olur. Head boyutu büyüdükçe score'lar büyür; softmax üstel bir fonksiyon olduğu için büyük farklar dağılımı tek bir token'a kilitler ve gradient'ler söner. $\sqrt{d_h}$'ye bölmek varyansı bire sabitliyor. İkinci işlem softmax: score'ların üstelini alıp toplamlarına bölüyoruz. Böylece her score pozitif bir weight'e dönüşüyor ve weight'lerin toplamı bir oluyor. Büyük score, büyük weight demek. Üçüncü işlem ağırlıklı toplam: value vektörlerini bu weight'lerle çarpıp topluyoruz. Sonuç y, bu token'ın geçmişten topladığı bilgi. Alttaki kod minGPT'den aynen alındı ve her satır bir formüle karşılık geliyor. İlk satır q ile k'nin transpozunun çarpımı ve ölçekleme; yani bütün score'lar tek bir matris çarpımıyla. İkinci satır maske; geleceği eksi sonsuz yapıyor. Üçüncü satır softmax. Dördüncü satır weight'lerle v'nin çarpımı. Kodun bütün diziyi aynı anda, matris çarpımlarıyla işlediğine dikkat: bu GPU için çok verimli. Donanımda ise generation sırasında her adımda tek bir token işleyeceğiz; bunu KV cache slaytında açacağım.

16. Elle bir örnek: üç geçmiş token, $d_h = 2$ ve x yönünde bir şey arayan bir query

Formülü somutlaştırmak için elle küçük bir örnek yapalım. Head boyutu 2 olsun ki her şeyi kafadan hesaplayabilelim. Şu anki token'ın query'si [1, 0]; yani x ekseni yönünde bir şey

arıyor. Geçmişte üç token var. Birincinin key'i [2, 0], tamamen x yönünde. İkincinin key'i [0, 2], tamamen y yönünde. Üçüncünün key'i [1, 1], ikisinin arasında. Birinci işlem score: query'yi her key ile dot product'a sokuyoruz. [1, 0] ile [2, 0]'ın dot product'ı 2; [0, 2] ile 0; [1, 1] ile 1. Bunları √2'ye bölüyoruz, çünkü head boyutu 2: 1,41, 0 ve 0,71 elde ediyoruz. İkinci işlem softmax: her birinin üstelini alıyoruz; 4,11, 1 ve 2,03. Toplamları 7,14. Her birini toplama bölünce weight'ler çıkıyor: 0,58, 0,14 ve 0,28. Toplamları tam olarak 1. Üçüncü işlem ağırlıklı toplam: value'ları bu weight'lerle çarpıp topluyoruz. Sonuç [0,86, 0,42]. Bu sonucu yorumlamak önemli. Query x yönünü arıyordu; en çok x yönlü key'e sahip birinci token'a baktı, ona 0,58 weight verdi. y yönlü ikinci token'a en az baktı, 0,14. Ortadaki üçüncü token ara bir pay aldı. Sonuç vektörü de ağırlıklı olarak birinci token'ın value'sunu taşıyor ama diğerlerinden de pay almış. İşte bir token'ın geçmişten "içeriğe göre" bilgi toplaması tam olarak bu. Gerçek modelde vektörler 16 boyutlu ve weight'ler training ile öğreniliyor, ama hesap bire bir aynı. Ayrıca bu tablo, attention'ın hesap maliyetini de gösteriyor: her geçmiş token için bir dot product, bir üstel, bir de value ile çarpma. Geçmiş uzadıkça bu iş doğrusal olarak artıyor.

17. Causal mask: her token yalnızca kendisine ve geçmişine bakabilir, gelecek tamamen gizlenir

Formülde j küçük eşit t koşulu vardı. Bu koşulu sağlayan mekanizmaya causal mask diyoruz. Soldaki tablo bütün query ve key pozisyon çiftlerini gösteriyor. Satırlar query pozisyonu, yani "şu an hangi token'dayım"; sütunlar key pozisyonu, yani "hangi token'a bakıyorum". Gri hücreler gelecek: j, t'den büyük. Bu hücrelerin score'u eksi sonsuz yapılıyor. Renkli hücreler ise softmax sonrası weight'ler; tonlar temsilî, gerçek değerler modele göre değişir. Her satır kendi içinde toplamı 1 olan bir dağılım. Neden böyle bir maske gerekli? Training sırasında model her pozisyonda bir sonraki token'ı tahmin etmeye çalışıyor. Eğer gelecekteki token'ları görebilseydi, cevap zaten önünde olurdu; kopyalardı ve hiçbir şey öğrenmezdi. Maske tam olarak nasıl çalışıyor? Score eksi sonsuz yapılınca softmax'taki üstel değer sıfır oluyor. Yani maskelenen weight yaklaşık olarak değil, tam olarak sıfır. Bunun güzel bir yan etkisi var: her pozisyon yalnızca kendi geçmişini gördüğü için, bir dizideki bütün pozisyonların tahmini tek bir forward geçişinde, birbirini bozmadan, paralel olarak yapılabiliyor. minGPT'nin training'i bu yüzden verimli. Ve en önemli sonuç alttaki cümle: t pozisyonundaki çıktı yalnızca 0'dan t'ye kadar olan token'lara bağlı. Yeni bir token geldiğinde eski pozisyonların hiçbir değeri değişmiyor. Bu, birkaç slayt sonra anlatacağım KV cache'in, yani generation'ı ucuzlatan fikrin matematiksel temeli. Kodda maske, torch.tril ile oluşturulan bir alt üçgen matris; bir kez oluşturulup modelin içinde saklanıyor.

18. Gerçek modelde: gpt-nano'nun bir head'i, çıktıyı yazarken girdideki ilgili rakamlara bakıyor

Şimdiye kadarki örnekler temsilîydi. Bu slayttaki sayılar ise gerçek: bizim eğittiğimiz gpt-nano modeline 2 0 1 1 0 2 girdisini verdik ve birinci layer'ın ilk head'inin softmax weight'lerini çıkardık. Tablonun her satırı bir pozisyon. Satırdaki sayılar, o pozisyonda bir sonraki token tahmin edilirken geçmişteki her pozisyona ne kadar weight verildiğini gösteriyor. Sağ üst üçgen gri, çünkü causal mask geleceği kapatıyor. Lacivert etiketler girdi token'ları, turuncu olanlar modelin ürettiği çıktı token'ları. İki satıra dikkatinizi çekmek istiyorum. Beşinci pozisyon girdinin son rakamı; burada model ilk çıktıyı, yani en küçük rakamı tahmin etmek zorunda. Bu head ağırlığının yüzde 96'sını girdideki iki 0'a veriyor: 1. pozisyondaki 0'a 0,36, 4. pozisyondaki 0'a 0,60. Model "en küçük rakamı yazacağım, girdide 0 var mı?" diye bakıyor gibi. Onuncu pozisyonda ise model son çıktıyı, yani en büyük rakamı tahmin ediyor. Bu kez ağırlığın yüzde 99'u girdideki iki 2'de: 0. pozisyon 0,31, 5. pozisyon 0,68. İşte attention'ın "içeriğe göre ağırlıklı ortalama" olmasının pratikteki anlamı bu: model hangi token'a bakacağını, o an ne yapması gerektiğine göre seçiyor. Önemli olan şu: bu davranışı biz programlamadık. Modele yalnızca sıralanmış örnekler gösterdik; bu bakma deseni training sırasında kendiliğinden ortaya çıktı. Dürüst olmak için şunu da söyleyeyim: diğer head'lerin desenleri bu kadar net değil. Sıralama işi tek bir head'de değil, üç layer ve dokuz head arasında paylaşılıyor; bu head en kolay okunanı.

19. Adım 5 · Multi-head attention: 48 boyut üç head'e bölünür ve her head kendi ilişkisini arar

Beşinci adım: multi-head attention. Tek bir attention, her pozisyonda tek bir ağırlıklı ortalama alabiliyor. Ama bir token'ın aynı anda birden çok şeye bakması gerekebilir. Örneğin sıralamada hem girdideki rakamları saymak hem de şimdiye kadar kaç rakam yazıldığını bilmek gerekiyor. Çözüm, attention'ı birkaç kez paralel çalıştırmak. minGPT bunu şöyle yapıyor: 48 boyutlu q, k ve v vektörlerini 16'şar boyutluk üç parçaya bölüyor. Her parça bir head. Her head, kendi 16 boyutlu q, k ve v'siyle kendi score'larını, kendi softmax weight'lerini ve kendi ağırlıklı toplamını bağımsız olarak hesaplıyor. Az önce gösterdiğim, girdideki rakamlara bakan head bunlardan biriydi. Sonra üç head'in 16 boyutlu çıktıları yan yana konuyor, yine 48 boyut oluyor ve c_proj adlı bir Linear katman bunları karıştırıyor. Böylece her head'in bulduğu bilgi tek bir vektörde birleşiyor. Maliyet açısından önemli bir nokta var: multi-head, hesabı artırmıyor. Toplam çarpma sayısı tek head'li, 48 boyutlu bir attention ile aynı; çünkü vektör yalnızca parçalara bölünüyor. Kodda bu, view ve transpose ile yapılan bir yeniden şekillendirme. view, 48 boyutu 3 çarpı 16 olarak yeniden yorumluyor; transpose da head boyutunu öne alıyor. Böylece PyTorch, head'leri sanki ayrı örneklermiş gibi tek bir matris çarpımında işliyor. İşlem bitince ters yönde birleştirip c_proj'dan geçiriyor. minGPT kodunda nh yerine self.n_head yazıyor; okunabilirlik için kısalttım.

20. Adım 6 · Residual connection: alt adımlar x'i silmez, üstüne bir düzeltme ekler

Altıncı adım residual connection. Bir block'un kodunun tamamı sağda: dört satır. Dikkat edilecek nokta, attention'ın ve MLP'nin sonucunun x'in yerine geçmemesi; x'in üstüne eklenmesi. Şekil bunu gösteriyor. x soldan sağa akan bir "residual stream". Attention, bu stream'in normalize edilmiş bir kopyasını okuyor, bir sonuç üretiyor ve sonucu stream'e ekliyor. Ardından MLP aynı şeyi yapıyor. Her block'ta iki ekleme var; üç block'lu modelde toplam altı. Bunun iki önemli sonucu var. Birincisi training ile ilgili. Formülde görüldüğü gibi, x artı f(x)'in türevi birim matris artı f'nin türevi. Yani türevde her zaman bir birim matris kalıyor. Bu sayede gradient, derin bir ağda her katmanda çarpılarak küçülmek yerine bu "kısa yol" üzerinden doğrudan geri akabiliyor. Residual connection olmadan derin transformer'ları eğitmek çok zor; bu fikir aslında görüntü işlemedeki ResNet'ten geliyor. İkincisi yorum ile ilgili. Residual stream'i bütün layer'ların ortak çalışma alanı gibi düşünebiliriz. Embedding bu alana token ve pozisyon bilgisini yazıyor. Her attention ve her MLP buradan okuyor ve buraya küçük bir düzeltme yazıyor. lm_head en sonda bu alanın son hâlini okuyor. Bu bakış açısı modelin ne yaptığını anlamak için çok kullanışlı. Donanım açısından da bir not: residual connection yalnızca bir toplama; ucuz bir işlem. Ama bu stream bütün model boyunca taşındığı için, sayıların hangi hassasiyette tutulacağı önemli bir tasarım kararı oluyor.

21. Adım 7 · MLP her token'ı kendi içinde işler: önce dört kat genişletir, GELU'dan geçirir, sonra daraltır

Yedinci adım MLP. Attention token'lar arasında bilgi taşıyordu; MLP ise her token'ın vektörünü kendi içinde, diğer token'lara bakmadan işliyor. Yapısı üstteki şeritte: normalize edilmiş 48 boyutlu vektör önce c_fc adlı Linear ile 192 boyuta, yani dört katına genişletiliyor. Sonra her elemana GELU adlı doğrusal olmayan fonksiyon uygulanıyor. Sonra c_proj adlı ikinci Linear ile tekrar 48 boyuta indiriliyor ve sonuç residual stream'e ekleniyor. Neden ortada doğrusal olmayan bir fonksiyon var? Çünkü iki Linear katman arka arkaya gelirse, matematiksel olarak tek bir Linear katmana eşittir: iki matrisin çarpımı yine bir matristir. Ortada bir non-linearity olmazsa genişletmenin hiçbir anlamı kalmaz. Modelin karmaşık ilişkileri temsil edebilmesi bu fonksiyonlar sayesinde. Sağdaki grafik GELU'yu gösteriyor; kesikli mavi çizgi karşılaştırma için ReLU. İkisi de pozitif değerleri neredeyse olduğu gibi geçiriyor, negatifleri sıfıra yakın bastırıyor. GELU'nun farkı, köşesinin yumuşak olması ve negatif tarafta küçük bir çukur bırakması; bu, türevin her yerde düzgün olmasını sağlıyor. minGPT, GPT-2 ile aynı tanh yaklaşımlı formülü kullanıyor; formül biraz karmaşık görünüyor ama tek girişli bir fonksiyon. Bu, donanım için önemli bir avantaj: tek girişli bir fonksiyon, giriş değerleri sınırlıysa önceden hesaplanmış bir tabloyla temsil edilebilir. Kodda MLP tek satırlık bir lambda: c_fc, GELU, c_proj ve dropout sırayla uygulanıyor. Bir de

parametre sayısı açısından not: MLP'nin iki Linear'ı, her block'taki parametrelerin yaklaşık üçte ikisini oluşturuyor.

22. Adım 8 · Son LayerNorm ve lm_head her token için bir logit üretir; generation bunlardan birini seçer

Son adım çıkış. Block'lardan çıkan vektör son bir LayerNorm'dan, ln_f'den geçiyor; ardından lm_head adlı Linear katman vocabulary'deki her token için bir skor üretiyor. Bu skorlara logit diyoruz. Sıralama görevinde vocabulary yalnızca üç token, 0, 1 ve 2; dolayısıyla üç logit çıkıyor. Logit'ler softmax'tan geçince toplamı bir olan olasılıklara dönüşüyor. Örnekte logit'ler 2,2, 0,4 ve eksi 0,8; olasılıklar 0,82, 0,14 ve 0,04. Değerler temsilî. Şimdi bu dağılımdan bir token seçmemiz gerekiyor. İki yol var. Birincisi greedy decoding: her zaman en olası token'ı seç. Burada önemli bir gözlem var: softmax monoton artan bir fonksiyon, yani sıralamayı değiştirmiyor. En büyük logit her zaman en büyük olasılığa karşılık geliyor. Bu yüzden greedy decoding'de softmax'ı hiç hesaplamadan doğrudan en büyük logit'i seçebiliriz. Donanım için bu güzel bir kolaylık, çünkü softmax'ta üstel fonksiyon ve bölme var. İkincisi sampling: dağılımdan rastgele çekmek. Bu durumda yüzde 82 ihtimalle 0, yüzde 14 ihtimalle 1 seçilir. Metin üretiminde çeşitlilik için sampling, sıralama gibi tek doğru cevabı olan görevlerde greedy kullanılıyor. Sağdaki kod minGPT'nin generate fonksiyonunun döngü gövdesi. Model çağrılıyor, son pozisyonun logit'leri alınıyor, temperature ile ölçekleniyor, softmax uygulanıyor, sonra sampling ya da greedy ile bir token seçilip dizinin sonuna ekleniyor. İlk satıra özellikle dikkat çekmek istiyorum: self(idx_cond). minGPT her yeni token için bütün diziyi, yani bütün prefix'i baştan modelden geçiriyor. Yüzüncü token'ı üretirken önceki doksan dokuz token'ı da yeniden hesaplıyor. Bu kodu basit tutuyor ama çok israflı. Bir sonraki slaytta bunun neden matematiksel olarak gereksiz olduğunu göstereceğim.

23. KV cache: geçmiş token'ların key ve value'ları hiç değişmediği için saklanıp yeniden kullanılır

Bu slayt, matematikten donanıma geçişin anahtarı. minGPT her yeni token için bütün diziyi baştan hesaplıyordu. Peki gerçekten gerekli mi? Hayır, ve nedeni causal mask. Hatırlayın: maske yüzünden j pozisyonundaki çıktı yalnızca 0'dan j'ye kadar olan token'lara bağlı. Bunu her layer için söyleyebiliriz; ispatı layer'lar üzerinden tümevarım. Embedding yalnızca o token'a ve pozisyona bakıyor. LayerNorm, Linear ve MLP her pozisyonda ayrı çalışıyor. Attention ise maske nedeniyle yalnızca kendinden önceki pozisyonların key ve value'larını okuyor; onlar da tümevarım hipotezine göre yalnızca kendi geçmişlerine bağlı. Sonuç: yeni bir token geldiğinde eski pozisyonların key ve value vektörleri hiç değişmiyor. O hâlde bunları her layer için saklayabiliriz; buna KV cache diyoruz. Şekil farkı gösteriyor. Soldaki tablo cache'siz durum: birinci adımda bir token, ikinci adımda iki token, beşinci adımda beş

token baştan hesaplanıyor; toplam 15 token hesabı. Sağdaki tablo cache'li durum: her adımda yalnızca yeni token hesaplanıyor, eskilerin key ve value'ları cache'ten okunuyor; toplam 5. Fark, dizi uzadıkça karesel ile doğrusal arasındaki fark. Bunun donanım açısından çok önemli bir sonucu var. Her adımda tek bir token işlendiği için modeldeki her Linear katman, matris-matris çarpımı yerine bir matris-vektör çarpımına iniyor. Attention ise t+1 uzunluğunda iki dot product dizisine iniyor: query ile saklanmış key'ler ve weight'ler ile saklanmış value'lar. Sağdaki pseudo-code bunun özü: yeni key ve value'yu sakla, query'yi saklanmış bütün key'lerle karşılaştır, weight'lerle saklanmış value'ları topla. minGPT'nin kodunda bu yok, ama modern bütün LLM'ler generation sırasında bunu yapıyor. Bizim FPGA tarafındaki her şey bu tek-token-adımı üzerine kurulacak.

24. FPGA, işlevi üretimden sonra tasarlanan bir çiptir ve hesap için dört tür yapı taşı sunar

Son bölüme, minGPT'nin FPGA'de uygulanma potansiyeline geçiyorum. Önce FPGA'nin ne olduğunu kısaca hatırlatayım. Bir işlemci ya da GPU'nun devresi fabrikada sabitlenir; biz onlara yalnızca komut veririz. FPGA'de ise devrenin kendisini, çip üretildikten sonra biz tasarlarız. Bunun için dört tür yapı taşı var. LUT'lar 6 girişli küçük doğruluk tabloları; her türlü mantık fonksiyonu bunlarla kuruluyor. Flip-flop'lar tek bitlik register'lar; ara sonuçları tutmak ve hesabı aşamalara bölmek, yani pipeline kurmak için kullanılıyor. BRAM'ler çip içindeki bellek blokları; bizim çipte 140 tane 36 kilobitlik blok var, toplam yaklaşık 630 kilobayt. DSP48'ler hazır çarpma ve toplama birimleri; her biri her clock cycle'da bir çarpma-toplama, yani bir MAC yapabiliyor. Bu sayılar projede kullandığımız PYNQ-Z2 kartındaki Zynq XC7Z020 çipine ait; küçük ve ucuz bir çip. Zynq'in güzel yanı, aynı çipte bir ARM işlemci ile programlanabilir mantığın birlikte bulunması. İşlemci Linux çalıştırıyor; modeli yükleyip sonuçları okuyabiliyor. Ağır hesap ise programlanabilir mantıkta yapılıyor. GPU ile farkı şöyle özetleyebilirim: GPU genel amaçlı ve çok güçlü, ama sabit çekirdeklerde komut yürütüyor. FPGA'de ise veri yolunu, belleğin düzenini ve paralelliği tam olarak modelin ihtiyacına göre biçimlendirebiliyoruz. Bu esneklik, minGPT gibi yapısı tamamen bilinen ve sabit bir hesap için çok değerli. Şimdi minGPT'nin iş yükünün bu yapı taşlarına nasıl oturduğuna bakalım.

25. minGPT'nin iş yükü: bir token'ın hesabının %96–98'i matris-vektör çarpımı ve model çip içine sığıyor

Bir donanım tasarlamadan önce iş yükünü sayılarla bilmek gerekiyor. Bu tablo, KV cache kullanıldığında gpt-nano'nun bir token için yaptığı hesabı gösteriyor. Her block'ta dört Linear katman var. q, k, v projection'ı 144 çarpı 48, yani 6 912 çarpma-toplama. Attention çıkışındaki c_proj 2 304. MLP'nin genişletme ve daraltma katmanları 9 216'şar. Bir block toplam 27 648; üç block ve küçük lm_head ile birlikte token başına 83 088 MAC.

Attention'ın kendi hesabı, yani score'lar ve ağırlıklı toplam, her layer'da 2 çarpı C çarpı t+1; pozisyona göre 96 ile 1 056 arasında değişiyor. Bu modelde context en fazla 11 token olduğu için attention toplamın yalnızca yüzde 2 ile 4'ü. Yani bir token'ın hesabının yüzde 96 ila 98'i matris-vektör çarpımı. Bu çok önemli bir bilgi: donanımın kalbi, hızlı bir matris-vektör çarpım birimi olmalı. DSP blokları tam olarak bunun için var. Sağdaki kutu bellek tarafı. gpt-nano'da 85,7 bin parametre var. Her biri 32 bitlik floating point olarak saklanırsa yaklaşık 343 kilobayt ediyor; bu bile çipin yaklaşık 630 kilobaytlık BRAM'ine sığıyor. 8 bite indirilirse 84 kilobayt; kapasitenin yalnızca yüzde 13'ü. Bunun anlamı şu: weight'ler için çipin dışındaki DRAM'e gitmemiz gerekmiyor. Dış bellek, FPGA tasarımlarında genellikle en büyük darboğazdır; bant genişliği sınırlıdır ve gecikmesi değişkendir. Model çipin içine sığdığı için bu sorun baştan ortadan kalkıyor. Bu iki gözlem, bir sonraki slayttaki güçlü yanların temeli.

26. Güçlü yanlar: minGPT'nin hesabı küçük, düzenli ve öngörülebilir; bu tam FPGA'nin sevdiği türden

Bu sayılardan sonra minGPT'nin FPGA için güçlü yanlarını altı maddede toplayabilirim. Birincisi: model kendini tekrar ediyor. Bütün layer'lar aynı hesabı farklı weight'lerle yapıyor. Bu yüzden her layer için ayrı donanım kurmak yerine tek bir veri yolu tasarlayıp layer'ları sırayla onun üzerinden geçirebiliriz; küçük bir çip için bu çok önemli. İkincisi: model çipe sığıyor. Weight'ler BRAM'de durabiliyor, dış DRAM'e gitmek gerekmiyor. Bu hem gecikmeyi düşürüyor hem de sabitliyor; çünkü dış bellek erişimi FPGA tasarımlarında genellikle hem yavaş hem de öngörülemez. Üçüncüsü: hesap tek tip. İşin yüzde 96 ila 98'i matris-vektör çarpımı. DSP bloklarından kurulmuş tek bir paralel çarpma-toplama dizisi bu işin neredeyse hepsini yapabilir. Dördüncüsü: akış tamamen öngörülebilir. Programda verinin değerine bağlı bir dallanma yok; her token, her seferinde aynı işlem sırasından geçiyor. Bu, donanımın zamanlamasını baştan, statik olarak planlayabileceğimiz anlamına geliyor; bir token'ın kaç clock cycle süreceğini hesap yapmadan bilebiliriz. Beşincisi: bazı parçalar donanımda neredeyse bedava. Embedding bir tablo okuması. Causal mask için hiçbir şey hesaplamak gerekmiyor; sadece j küçük eşit t olan konumları okumak yetiyor. GELU tek girişli bir fonksiyon olduğu için tabloyla gerçeklenebilir. Altıncısı: batch boyutu bir olduğunda, yani tek bir kullanıcı için tek tek token üretirken, GPU'lar kapasitelerinin küçük bir kısmını kullanabiliyor; çünkü verimleri büyük batch'lerden geliyor. FPGA'de ise veri yolunu tam bu tek-token durumuna göre tasarlayabiliyoruz. Özetle: küçük bir FPGA, minGPT ölçeğindeki bir modeli dış belleğe ihtiyaç duymadan, düşük ve sabit gecikmeyle çalıştırmak için çok uygun.

27. Zayıf yanlar: non-linear işlemler, büyüyen attention ve floating point için yazılmış bir model

Madalyonun öbür yüzü, zayıf yanlar. Bunları da altı maddede topladım. Birincisi: non-linear işlemler. Matris-vektör çarpımı donanımda kolay, ama LayerNorm'da karekök ve bölme, softmax'ta üstel fonksiyon ve bölme var. Bunlar donanımda pahalı ve yavaş; her biri için özel bir hesap birimi tasarlamak gerekiyor. İşin yüzde 96'sından fazlası çarpma-toplama olsa da, tasarımın en zor kısmı bu küçük yüzde. İkincisi: floating point. Model bilgisayarda 32 bitlik floating point sayılarla eğitiliyor. FPGA'de floating point aritmetik hem çok kaynak tüketiyor hem yavaş. Doğal çözüm, sayıları daha düşük bit genişliğine indirmek; ama bunu yaparken modelin doğruluğunu korumak ayrı bir mühendislik problemi. Üçüncüsü: attention'ın maliyeti pozisyonla büyüyor. Her yeni token, saklanan bütün key'lerle karşılaştırılıyor. gpt-nano'da context 11 token olduğu için bu önemsiz, ama context yüzlerce token'a çıkınca attention darboğaz olmaya başlıyor. Dördüncüsü: KV cache belleği layer sayısı, context uzunluğu ve vektör boyutuyla çarpımsal büyüyor. minGPT her head için ayrı key ve value tutuyor; modern modellerdeki GQA bu belleği azaltıyor. Beşincisi: minGPT'nin kodu GPU için yazılmış. Bütün diziyi tek seferde, matris-matris çarpımlarıyla işliyor ve KV cache kullanmıyor. Bu yüzden kodu doğrudan donanıma çeviremiyoruz; token başına çalışan bir hesaba yeniden yazmamız gerekiyor. Altıncısı: büyümeye sınır. Model büyüdükçe weight'ler çipe sığmaz ve dış belleğe gitmek gerekir; o zaman sınırı hesap gücü değil, bellek bant genişliği belirler. Ayrıca learned position embedding, context uzunluğunu training sırasındaki block_size ile sabitliyor. Ama alttaki özet önemli: bu zorlukların hepsi önceden belli. Hangi işlemin pahalı olduğunu, nerede yeniden yazmak gerektiğini biliyoruz. Bu da minGPT'yi FPGA'de bir language model çalıştırmak için çok iyi bir başlangıç noktası yapıyor.

28. son

Toparlamak için üç cümle. Birincisi: minGPT yeni bir mimari değil, GPT-2'nin sade ve okunabilir hâli. Embedding, her biri attention ve MLP'den oluşan L tane aynı block ve bir lm_head; hepsi residual connection ile birbirine bağlı. Bu çekirdek, bugünkü büyük LLM'lerin de çekirdeği. İkincisi: modelin kalbi attention ve attention aslında basit bir fikir: geçmişteki token'ların, ağırlıkları içeriğe göre hesaplanan bir ortalaması. Causal mask sayesinde geçmiş token'ların değerleri hiç değişmiyor; bu da KV cache'i mümkün kılıyor ve generation sırasında her adımda yalnızca tek bir token işlemek yetiyor. Üçüncüsü: bu yapı bir FPGA için çok uygun. Model küçük, kendini tekrar ediyor, hesabın neredeyse tamamı matris-vektör çarpımı ve akış tamamen öngörülebilir. Zorluklar da belli: karekök, üstel fonksiyon ve bölme gibi non-linear işlemler, modelin floating point ile eğitilmiş olması ve uzun context'te büyüyen attention maliyeti. Bir sonraki bölümde bu potansiyeli gerçekleştirdiğimiz FPGA tasarımını anlatacağım. Teşekkür ederim; sorularınızı bekliyorum.

Ek · Olası sorular ve kısa cevaplar

Skorları neden $\sqrt{d_h}$'ye bölüyoruz? q ve k'nin bileşenleri bağımsız, ortalaması 0 ve varyansı 1 ise dot product d_h tane çarpımın toplamıdır; varyansı d_h olur. Head boyutu büyüdükçe score'lar büyür, softmax tek bir token'a kilitlenir ve gradient'ler söner. $\sqrt{d_h}$'ye bölmek varyansı 1'e sabitler. gpt-nano'da $d_h = 16$, yani çarpan 1/4.

Pre-LN nedir, neden kullanılıyor? LayerNorm, attention ve MLP'ye girmeden önce uygulanıyor; residual yolun kendisi normalize edilmiyor. Böylece residual stream üzerinden gradient doğrudan akabiliyor ve derin modellerde training daha kararlı oluyor. GPT-2 ve minGPT bu düzeni kullanıyor.

MLP neden dört kat genişliyor? Orijinal Transformer makalesinden gelen deneysel bir tercih. Sonucu şu: her block'taki parametrelerin yaklaşık üçte ikisi MLP'de. gpt-nano'da bir block 28 272 parametre; bunun 18 672'si MLP.

Attention'ın maliyeti neden "karesel" deniyor? T token'lık bir dizide her token kendinden önceki bütün token'larla karşılaştırılıyor; toplam yaklaşık $T^2/2$ dot product. KV cache ile generation sırasında token başına maliyet doğrusal: t'inci token t+1 key ile karşılaştırılıyor.

KV cache ne kadar bellek ister? Her layer'da her pozisyon için bir key ve bir value vektörü: toplam $2 \cdot L \cdot T \cdot C$ sayı. gpt-nano'da $2 \times 3 \times 11 \times 48 = 3\,168$ sayı; çok küçük. Uzun context'li ve çok layer'lı modellerde ise belleğin önemli bir kısmını kaplıyor.

Greedy decoding'de softmax neden gerekmiyor? Softmax monoton artan bir fonksiyon; logit'lerin sırasını değiştirmiyor. En büyük logit her zaman en büyük olasılığa karşılık geliyor. Bu yüzden greedy decoding'de doğrudan en büyük logit seçilebilir.

minGPT ile GPT-2 arasındaki fark yalnızca ölçek mi? Mimari aynı; minGPT, OpenAI'nin GPT-2 weight'lerini from_pretrained ile yükleyip çalıştırabiliyor. Farklar parametre sayısı, training verisi ve kodun hız için optimize edilmemiş olması (KV cache yok, FlashAttention yok).

Model sıralamayı gerçekten "anlıyor" mu? Model sıralama algoritması bilmiyor; altı rakamlık dizilerde bir sonraki token'ın dağılımını öğreniyor. 729 girdinin hepsinde doğru sonuç veriyor ve attention weight'lerinde girdideki ilgili rakamlara baktığını görebiliyoruz. Ama başka uzunluklarda test edilmedi; context de block_size ile sınırlı.

Neden FPGA, neden GPU değil? GPU'lar büyük batch'lerde çok yüksek throughput sağlar. Tek kullanıcı ve tek token'lık generation'da kapasitelerinin küçük bir kısmını kullanırlar. FPGA'de veri yolu tam bu duruma göre tasarlanabiliyor: düşük ve sabit gecikme, düşük güç ve gömülü sistemlere uygunluk. Bu, minGPT ölçeğindeki küçük modeller için anlamlı bir hedef.

Training'i de FPGA'de yapabilir miyiz? Prensipte evet, ama backpropagation, optimizer durumu ve yüksek hassasiyetli aritmetik gerekir; donanım çok daha karmaşık olur. Bu projenin kapsamı inference: model bilgisayarda eğitiliyor, FPGA'de çalıştırılıyor.